University Management System

Technical Explanation of Project Structure, Docker Files, and REST API

A line-by-line reference for the Spring Boot application in
C:\code\project\software

Application	Spring Boot 3.4.5 REST API
Language / Runtime	Java 21
Security	JWT authentication, BCrypt password hashing, role-based access control
Database	H2 for local development; PostgreSQL for Docker Compose
Containerization	Dockerfile and compose.yaml
Document generated	2026-10-26 00:01

Contents

1. Files and folders explanation
2. Dockerfile: keyword by keyword
3. Dockerfile: line by line
4. compose.yaml: keyword by keyword
5. compose.yaml: line by line
6. .dockerignore: pattern by pattern
7. .dockerignore: line by line
8. REST API: where it is used
9. REST API endpoint map and request flow
10. REST controllers: line by line
11. Request and response DTOs: line by line
12. REST security route rules: line by line
13. JWT request filter: line by line
14. Supporting REST classes and HTTP status summary

How to read line numbers: All line numbers in this PDF refer to the source files as they existed when this document was generated. Blank lines are included so the numbering stays aligned with the actual files.

1. Files and folders explanation

The project is a standard Maven Spring Boot layout. Hand-written application code lives under src/main. Tests live under src/test. Maven writes generated output to target. Root-level files support building, containerization, configuration, and documentation.

Project tree

```
software/
|-- .mvn/
|   `-- wrapper/
|       `-- maven-wrapper.properties
|-- src/
|   |-- main/
|   |   |-- java/com/example/university/
|   |   |   |-- admin/
|   |   |   |-- auth/
|   |   |   |-- config/
|   |   |   |-- error/
|   |   |   |-- security/
|   |   |   |-- student/
|   |   |   |-- teacher/
|   |   |   |-- user/
|   |   |   `-- UniversityManagementApplication.java
|   |   `-- resources/
|   |       |-- application.yml
|   |       `-- application-docker.yml
|   `-- test/
|       `-- java/com/example/university/
|           `-- UniversityManagementApplicationTests.java
|-- target/                         (generated by Maven)
|-- .aider.chat.history.md
|-- .aider.input.history
|-- .dockerignore
|-- .env.example
|-- .gitignore
|-- compose.yaml
|-- Dockerfile
|-- mvnw
|-- mvnw.cmd
|-- pom.xml
`-- README.md
```

Root files

File or folder	Purpose
pom.xml	Maven build file. Declares Java 21, Spring Boot 3.4.5, Web, Security, JPA, Validation, Actuator, JWT, H2, PostgreSQL, and tests.
mvnw	Maven Wrapper launcher for Linux/macOS. Downloads and runs the pinned Maven version.
mvnw.cmd	Maven Wrapper launcher for Windows.
.mvn/wrapper/maven-wrapper.properties	Pins Maven 3.9.9 and selects the wrapper distribution type.
Dockerfile	Multi-stage container build. Builds the JAR with Maven, then runs it in an Alpine Java 21 JRE image.
compose.yaml	Docker Compose definition for the Spring Boot app and PostgreSQL, including health checks and environment wiring.
.dockerignore	Excludes build output, IDE files, Git data, secrets, and the README from the Docker build context.
.env.example	Template for database credentials, JWT settings, bootstrap passwords, and the host port.
.gitignore	Excludes generated/IDE/local-secret files from Git if the project is placed in a repository.
README.md	Human-readable setup, API, role, configuration, testing, and security documentation.
.aider.chat.history.md	Aider tool conversation history. It is not part of the application and can be deleted.
.aider.input.history	Aider tool input history. It is unrelated to the application and can be deleted.

Generated folder

target/ is created by Maven and should not be edited. It contains compiled main classes, compiled test classes, Maven status metadata, generated source folders, Surefire test reports, app.jar (the runnable Spring Boot JAR), and app.jar.original (the pre-repackaging JAR). The entire folder may be deleted and regenerated with a build.

src/main/resources

File	Purpose
application.yml	Default local profile. Uses in-memory H2, defines the HTTP port, JWT settings, Actuator exposure, and default bootstrap accounts.
application-docker.yml	Docker profile. Overrides the datasource with PostgreSQL connection values taken from environment variables.

src/test/java

UniversityManagementApplicationTests.java contains nine integration tests. They use MockMvc to check login, invalid credentials, role restrictions, admin access to all resources, CRUD, teacher-created role assignment, token revocation after logout, and validation errors.

Java source files

The application is grouped by responsibility. Controllers expose HTTP endpoints, security classes protect them, user classes hold shared account logic, and DTOs define the JSON contract.

Java file	Responsibility
UniversityManagementApplication.java	Application entry point with @SpringBootApplication and main().
admin/AdminController.java	CRUD endpoints for admin accounts at /api/admins.
teacher/TeacherController.java	CRUD endpoints for teacher accounts at /api/teachers.
student/StudentController.java	CRUD endpoints for student accounts at /api/students.
auth/AuthController.java	Login, logout, and current-user endpoints under /api/auth.
auth/CustomUserDetailsService.java	Loads a database account for Spring Security login.
auth/LoginRequest.java	JSON request record for login credentials.
auth/LoginResponse.java	JSON response record containing the JWT and safe user data.
auth/UserPrincipal.java	Authenticated user id, username, and role stored in the security context.
auth/InvalidCredentialsException.java	Maps failed login to HTTP 401.
config/DataInitializer.java	Creates missing bootstrap ADMIN, TEACHER, and STUDENT accounts on startup.
error/ApiError.java	Consistent JSON error response record.
error/GlobalExceptionHandler.java	Converts validation, not-found, conflict, credential, and unexpected exceptions into JSON status responses.
security/SecurityConfig.java	Password encoder, JWT filter registration, stateless sessions, and route-to-role rules.
security/JwtService.java	Creates and verifies signed JWT access tokens.
security/JwtAuthenticationFilter.java	Reads Bearer tokens, validates them, loads the current database user, and authenticates the request.
security/RestAuthenticationEntryPoint.java	Returns a JSON 401 response for unauthenticated requests.
security/RestAccessDeniedHandler.java	Returns a JSON 403 response for authenticated requests without the required role.
user/Role.java	Enum of ADMIN, TEACHER, STUDENT.
user/User.java	JPA entity for every account, including username, BCrypt hash, role, enabled flag, and token version.
user/UserRepository.java	Spring Data JPA queries by username, role, and id.
user/UserAccountService.java	Shared CRUD service: role filtering, username normalization, hashing, duplicate checks, and token revocation.
user/UserWriteRequest.java	Validated JSON body for creating or updating a user.
user/UserResponse.java	Safe JSON response shape that omits the password hash.
user/InvalidRequestException.java	Maps business validation failures to HTTP 400.
user/ResourceNotFoundException.java	Maps missing resources to HTTP 404.
user/UsernameAlreadyExistsException.java	Maps duplicate usernames to HTTP 409.

Important configuration files outside the Java source

File	Meaning
.env.example	Shows PostgreSQL credentials, APP_PORT, JWT secret, token lifetime, and bootstrap account settings.
.mvn/wrapper/maven-wrapper.properties	Keeps the build reproducible by pinning Maven 3.9.9.
.gitignore	Prevents target/, IDE metadata, logs, and .env from being tracked by Git.

2. Dockerfile: keyword by keyword

The Dockerfile uses a multi-stage build. The first stage compiles the project. The second stage contains only the runtime and the finished JAR, keeping the final image smaller and avoiding Maven/source code in production.

Keyword / token	Meaning	Where it appears
FROM	Starts a new build stage and selects the base image.	
maven:3.9.9-eclipse-temurin-21	Maven 3.9.9 plus a Java 21 JDK, used only for building.	
AS build	Names this stage 'build' so a later stage can copy from it.	
WORKDIR	Sets the working directory for following instructions.	
COPY	Copies files from the build context or from another stage into the image.	
RUN	Executes a command while building the image. It becomes an image layer.	
mvn -B -q	Runs Maven in batch mode (-B) and quiet mode (-q).	
dependency:go-offline	Resolves and caches project dependencies so later builds can reuse them.	
-DskipTests	Compiles and packages without running tests inside the image build.	
eclipse-temurin:21-jre-alpine	Small Alpine Linux image with a Java 21 runtime only, no compiler or Maven.	
COPY --from=build	Copies an artifact from the previously named build stage.	
/workspace/target/app.jar	The executable JAR produced by the Maven package step.	
USER	Switches the container process to a non-root user.	
EXPOSE	Documents the port the application listens on. It does not publish the port by itself.	
HEALTHCHECK	Defines a command Docker can run to decide whether the container is healthy.	
--interval	How often the health check runs.	
--timeout	Maximum time allowed for one health-check attempt.	
--start-period	Grace period after startup before failed checks count against the container.	
--retries	Number of consecutive failures before the container is marked unhealthy.	
CMD	Default command for the health check; also the runtime command form when used normally.	
wget -qO-	Fetches a URL quietly and writes the response body to stdout.	
|| exit 1	Shell OR operator. If wget fails, exit with failure so the health check fails.	
ENTRYPOINT	Sets the main executable of the container.	
-XX:MaxRAMPercentage=75.0	Caps the Java heap at about 75% of the container memory limit.	
JSON exec form	The bracket array form runs arguments directly without an extra shell.	
\ (backslash)	Continues one Dockerfile instruction onto the next physical line.	

3. Dockerfile: line by line

The following table explains every physical line of Dockerfile.

Line	Source line	Explanation
1	`FROM maven:3.9.9-eclipse-temurin-21 AS build`	Starts the build stage from the Maven 3.9.9 Java 21 image and names this stage 'build'.
2	`WORKDIR /workspace`	Sets /workspace as the working directory inside the build stage.
3		Blank separator line.
4	`COPY pom.xml .`	Copies pom.xml from the project into /workspace as pom.xml.
5	`RUN mvn -B -q dependency:go-offline`	Downloads and caches Maven dependencies in batch/quiet mode before source code is copied, improving layer caching.
6		Blank separator line.
7	`COPY src ./src`	Copies the source tree into /workspace/src.
8	`RUN mvn -B -q package -DskipTests`	Packages the application into /workspace/target/app.jar without running tests.
9		Blank separator line.
10	`FROM eclipse-temurin:21-jre-alpine`	Starts the final runtime stage from the small Java 21 Alpine JRE image.
11	`WORKDIR /app`	Sets /app as the runtime working directory.
12		Blank separator line.
13	`RUN addgroup -S app && adduser -S app -G app`	Creates a system group 'app' and a system user 'app' in that group.
14	`COPY --from=build /workspace/target/app.jar app.jar`	Copies only the application JAR from the build stage into /app/app.jar.
15		Blank separator line.
16	`USER app`	Runs the JAR as the non-root app user.
17	`EXPOSE 8080`	Documents that the application listens on port 8080.
18		Blank separator line.
19	`HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \`	Starts the container health-check configuration with interval, timeout, startup grace period, and retry count.
20	`    CMD wget -qO- http://127.0.0.1:8080/actuator/health || exit 1`	Health check runs wget against Actuator health; failure returns exit code 1.
21		Blank separator line.
22	`ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "/app/app.jar"]`	Starts the JAR with Java, caps heap usage at 75% of container memory, and uses /app/app.jar.

Why there are two FROM instructions: The first FROM creates a temporary build environment. The second FROM creates the much smaller final runtime environment. Only the finished JAR crosses between stages.

4. compose.yaml: keyword by keyword

compose.yaml describes a multi-container application. Docker Compose creates the db and app containers on one private network, waits for PostgreSQL to become healthy, and passes configuration through environment variables.

Keyword / token	Meaning	Where it appears
services	Top-level map of containers Compose should create.	
db	Name of the PostgreSQL service. Other services can reach it by this hostname on the Compose network.	
app	Name of the Spring Boot service.	
image	Uses a prebuilt container image instead of building one.	
postgres:16-alpine	PostgreSQL 16 running on a small Alpine Linux base image.	
environment	Defines environment variables for the container. Here it is a KEY: value map.	
${NAME:-default}	Compose variable substitution: use host value NAME, otherwise use default.	
volumes	Service-level list of mounts, or top-level declaration of named volumes.	
postgres-data:/var/lib/postgresql/data	Mounts the named volume postgres-data into PostgreSQL's data directory.	
healthcheck	Defines how Compose decides whether the database container is healthy.	
test	Command used by the health check.	
CMD-SHELL	Runs the test through a shell, allowing commands, flags, and variables.	
$${POSTGRES_USER}	Double dollar escapes Compose interpolation. The container shell receives ${POSTGRES_USER}.	
pg_isready	PostgreSQL utility that checks whether the server accepts connections.	
interval	Delay between health-check attempts.	
timeout	Maximum duration of one health-check attempt.	
retries	Number of failed attempts before the service is unhealthy.	
restart	Container restart policy.	
unless-stopped	Restart on failure or daemon restart, except when explicitly stopped.	
build	Tells Compose to build an image from a Dockerfile.	
context	Build context directory sent to the Docker daemon.	
depends_on	Controls startup ordering between services.	
condition: service_healthy	Wait until the db health check reports healthy before starting the app.	
ports	Publishes container ports to the host.	
HOST:CONTAINER	The left side is host port; the right side is container port.	
volumes: (top level)	Declares named volumes managed by Compose.	
postgres-data:	Creates a persistent named volume so database data survives container recreation.	
YAML indentation	Spaces define nesting. Consistent indentation is required.	
double quotes	Used here to protect strings containing substitution syntax, punctuation, or special characters.	
hyphen list	A leading '-' marks an item in a YAML list, such as a volume mount or port mapping.	

Modern Compose: No top-level version: key is required in current Docker Compose. The file starts directly with services. The app connects to PostgreSQL using DB_HOST=db because Compose provides service-name DNS.

5. compose.yaml: line by line

Every physical line of compose.yaml is explained below.

Line	Source line	Explanation
1	`services:`	Starts the top-level services map.
2	`  db:`	Declares the PostgreSQL service named db.
3	`    image: postgres:16-alpine`	Uses the PostgreSQL 16 Alpine image.
4	`    environment:`	Starts the database environment-variable map.
5	`      POSTGRES_DB: "${POSTGRES_DB:-university}"`	Sets the database name from POSTGRES_DB, or university when the host variable is absent.
6	`      POSTGRES_USER: "${POSTGRES_USER:-university}"`	Sets the database user from POSTGRES_USER, or university.
7	`      POSTGRES_PASSWORD: "${POSTGRES_PASSWORD:-university}"`	Sets the database password from POSTGRES_PASSWORD, or university.
8	`    volumes:`	Starts the list of database volume mounts.
9	`      - postgres-data:/var/lib/postgresql/data`	Mounts the named volume postgres-data at PostgreSQL's data directory for persistence.
10	`    healthcheck:`	Starts the database health-check settings.
11	`      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]`	Runs pg_isready inside the container; $$ escapes Compose so the container shell expands its own variables.
12	`      interval: 10s`	Checks the database every 10 seconds.
13	`      timeout: 5s`	Allows each check up to 5 seconds.
14	`      retries: 5`	Marks the database unhealthy after 5 consecutive failures.
15	`    restart: unless-stopped`	Restarts the database unless it was explicitly stopped.
16		Blank separator line.
17	`  app:`	Declares the Spring Boot application service named app.
18	`    build:`	Starts build configuration for the app image.
19	`      context: .`	Uses the current project directory as the Docker build context.
20	`    depends_on:`	Starts the app's dependency configuration.
21	`      db:`	Depends on the db service.
22	`        condition: service_healthy`	Waits for db to pass its health check before starting app.
23	`    environment:`	Starts the app environment-variable map.
24	`      SPRING_PROFILES_ACTIVE: docker`	Activates the Spring profile named docker, loading application-docker.yml.
25	`      DB_HOST: db`	Points the app at the db service hostname.
26	`      DB_PORT: "5432"`	Sets PostgreSQL port 5432 as a string.
27	`      DB_NAME: "${POSTGRES_DB:-university}"`	Copies the database name to the app's DB_NAME variable.
28	`      DB_USERNAME: "${POSTGRES_USER:-university}"`	Copies the database user to DB_USERNAME.
29	`      DB_PASSWORD: "${POSTGRES_PASSWORD:-university}"`	Copies the database password to DB_PASSWORD.
30	`      JWT_SECRET: "${JWT_SECRET:-development-only-jwt-secret-change-me-0123456789abcdef}"`	Sets the JWT signing secret from the host or uses the development default.
31	`      JWT_EXPIRATION_MS: "${JWT_EXPIRATION_MS:-3600000}"`	Sets JWT lifetime to 3600000 ms (one hour) by default.
32	`      BOOTSTRAP_ENABLED: "${BOOTSTRAP_ENABLED:-true}"`	Enables creation of missing bootstrap accounts by default.
33	`      BOOTSTRAP_ADMIN_USERNAME: "${BOOTSTRAP_ADMIN_USERNAME:-admin}"`	Sets the bootstrap admin username.
34	`      BOOTSTRAP_ADMIN_PASSWORD: "${BOOTSTRAP_ADMIN_PASSWORD:-ChangeMe123!}"`	Sets the bootstrap admin password; change this outside a demo.
35	`      BOOTSTRAP_TEACHER_USERNAME: "${BOOTSTRAP_TEACHER_USERNAME:-teacher}"`	Sets the bootstrap teacher username.
36	`      BOOTSTRAP_TEACHER_PASSWORD: "${BOOTSTRAP_TEACHER_PASSWORD:-ChangeMe123!}"`	Sets the bootstrap teacher password; change this outside a demo.
37	`      BOOTSTRAP_STUDENT_USERNAME: "${BOOTSTRAP_STUDENT_USERNAME:-student}"`	Sets the bootstrap student username.
38	`      BOOTSTRAP_STUDENT_PASSWORD: "${BOOTSTRAP_STUDENT_PASSWORD:-ChangeMe123!}"`	Sets the bootstrap student password; change this outside a demo.
39	`    ports:`	Starts host-to-container port publishing.
40	`      - "${APP_PORT:-8080}:8080"`	Maps host port APP_PORT (default 8080) to container port 8080.
41	`    restart: unless-stopped`	Restarts the app unless explicitly stopped.
42		Blank separator line.
43	`volumes:`	Starts the top-level named-volume declarations.

Line	Source line	Explanation
44	`  postgres-data:`	Declares the postgres-data volume used by the database service.

Startup sequence: Compose creates the network and volume, starts db, runs its health check, then starts app. The application uses the Docker Spring profile and connects to jdbc:postgresql://db:5432/university.

6. .dockerignore: pattern by pattern

.dockerignore is not a programming language. Each line is a path pattern relative to the Docker build context. Docker excludes matching paths before sending the context to the daemon, making builds faster and preventing secrets or generated files from entering the image build.

Pattern concept	Meaning
One pattern per line	Each entry is matched against paths under the build context.
Relative path	Patterns are relative to the context directory, not to the Dockerfile's absolute location.
Trailing slash /	Matches a directory and its contents, such as target/ or .git/.
Exact filename	Matches that path, such as .env or README.md.
Glob *	Matches any characters within a path segment. *.iml matches any file ending in .iml.
No negation (!) here	This file only excludes paths. It does not re-include anything.
Secret protection	.env is excluded so local secrets are not sent as build context.
Build output	target/ is excluded because Maven output is generated during the Docker build.
Hidden directories	.git/, .idea/, and .vscode/ are excluded to reduce context size and avoid local metadata.

.dockerignore: line by line

Line	Source line	Explanation
1	`target/`	Excludes the entire Maven target/ directory and its generated contents.
2	`.git/`	Excludes Git metadata so repository internals are not sent to the Docker build.
3	`.gitignore`	Excludes the root .gitignore file itself.
4	`.idea/`	Excludes IntelliJ IDEA project metadata.
5	`.vscode/`	Excludes Visual Studio Code workspace metadata.
6	`*.iml`	Excludes all IntelliJ .iml module files anywhere in the context.
7	`.env`	Excludes the local .env file, which may contain real secrets.
8	`README.md`	Excludes README.md from the build context; documentation is not needed inside the image.

Important distinction: .dockerignore affects only the Docker build context. It does not stop a file from existing in the repository, and it does not control what Docker Compose mounts at runtime.

8. REST API: where it is used

The REST API is implemented by the controller classes. A controller is a Spring bean annotated with @RestController; Spring's MVC/Jackson integration converts its Java objects into JSON. Security sits in front of the controllers, and the service layer performs the database work.

File	Lines	How it participates in the REST API
AdminController.java	Lines 22-67	Defines full CRUD for /api/admins. Route is restricted to ADMIN.
TeacherController.java	Lines 22-67	Defines full CRUD for /api/teachers. Route is allowed for ADMIN and TEACHER.
StudentController.java	Lines 22-67	Defines full CRUD for /api/students. Route is allowed for ADMIN and STUDENT.
AuthController.java	Lines 24-81	Defines login, logout, and current-user endpoints under /api/auth.
LoginRequest.java / LoginResponse.java	All lines	Define the JSON login request and token response.
UserWriteRequest.java	All lines	Defines and validates the JSON body used by create and update endpoints.
UserResponse.java	All lines	Defines the safe JSON user response; it omits the password hash.
ApiError.java	All lines	Defines the consistent JSON error response shape.
GlobalExceptionHandler.java	Methods	Maps exceptions to HTTP status codes and JSON error bodies.
SecurityConfig.java	Lines 47-56	Applies public/protected route rules and role checks before controllers run.
JwtAuthenticationFilter.java	Lines 43-82	Extracts and verifies Bearer tokens for each API request.
UserAccountService.java	CRUD methods	Performs role-filtered CRUD, BCrypt hashing, validation, and token revocation.

Important REST annotations

Annotation	REST purpose
@RestController	Combines @Controller and @ResponseBody. Returned objects become JSON responses.
@RequestMapping("/api/...")	Defines the common base URL for all methods in a controller.
@GetMapping	Handles HTTP GET reads. Should be safe and idempotent.
@PostMapping	Handles HTTP POST creation and returns 201 with a Location header for resources.
@PutMapping	Handles full update of an existing resource.
@DeleteMapping	Handles deletion and returns 204 No Content on success.
@PathVariable	Reads a value from the URL, such as the {id} in /api/students/5.
@RequestBody	Deserializes the JSON request body into a Java record.
@Valid	Runs Bean Validation constraints on the request DTO before the controller body executes.
ResponseEntity<T>	Represents the full HTTP response: status, headers, and JSON body.

Where each HTTP status is produced

Status	Where it is used
200 OK	GET list, GET single, successful PUT, login, and current-user responses.
201 Created	POST handlers. The Location header points to the new resource.
204 No Content	DELETE handlers and successful logout.
400 Bad Request	Invalid JSON, failed Bean Validation, or invalid business request.
401 Unauthorized	Bad login credentials or missing/invalid/revoked JWT.
403 Forbidden	Authenticated user has the wrong role for the route.
404 Not Found	Id does not exist or does not belong to the requested role.
409 Conflict	Duplicate username or conflicting database data.
415 Unsupported Media Type	Request body is not sent as a supported content type.
500 Internal Server Error	Unexpected server-side failure; details are logged, not exposed.

9. REST API endpoint map and request flow

Endpoint map

Method	Path	Allowed role	Result
POST	/api/auth/login	Public	Validate credentials and return a JWT.
POST	/api/auth/logout	Authenticated	Revoke all existing tokens for the current account.
GET	/api/auth/me	Authenticated	Return the current account.
GET	/api/admins	ADMIN	List admins.
GET	/api/admins/{id}	ADMIN	Read one admin.
POST	/api/admins	ADMIN	Create an admin; 201 + Location.
PUT	/api/admins/{id}	ADMIN	Update an admin; 200.
DELETE	/api/admins/{id}	ADMIN	Delete an admin; 204.
GET	/api/teachers	ADMIN, TEACHER	List teachers.
GET	/api/teachers/{id}	ADMIN, TEACHER	Read one teacher.
POST	/api/teachers	ADMIN, TEACHER	Create a teacher; 201 + Location.
PUT	/api/teachers/{id}	ADMIN, TEACHER	Update a teacher; 200.
DELETE	/api/teachers/{id}	ADMIN, TEACHER	Delete a teacher; 204.
GET	/api/students	ADMIN, STUDENT	List students.
GET	/api/students/{id}	ADMIN, STUDENT	Read one student.
POST	/api/students	ADMIN, STUDENT	Create a student; 201 + Location.
PUT	/api/students/{id}	ADMIN, STUDENT	Update a student; 200.
DELETE	/api/students/{id}	ADMIN, STUDENT	Delete a student; 204.

Request flow through the application

- The client sends an HTTP request. For protected routes it includes Authorization: Bearer <jwt>.
- Spring Security's filter chain runs. JwtAuthenticationFilter reads the header and validates the token.
- The filter loads the current account from UserRepository and compares the token version with the account's token version.
- SecurityConfig applies URL rules such as hasRole('ADMIN') or hasAnyRole('ADMIN','TEACHER').
- Spring MVC matches the HTTP method and path to a controller method.
- @RequestBody converts JSON into a DTO, and @Valid checks field constraints.
- The controller calls UserAccountService, which applies role filtering, duplicate checks, and BCrypt hashing.
- The service uses UserRepository and JPA to read or write PostgreSQL/H2.
- The controller returns ResponseEntity: status, optional headers, and a DTO.
- Jackson serializes the DTO to JSON. Password hashes are never present in UserResponse.

REST API is used only in the web layer: Controllers should not contain database logic. The shared UserAccountService owns business rules, and repositories own persistence. This keeps the HTTP API consistent across all three resources.

10. REST controllers: line by line

The admin, teacher, and student controllers have the same CRUD shape. The differences are the package, class name, base URL, and fixed role. Each row below corresponds to a real source line.

Admin REST controller

Admin-only CRUD. Admins can also use the teacher and student controllers because SecurityConfig grants ADMIN access to those routes.

Line	Source line	Explanation
1	`package com.example.university.admin;`	Declares the Java package com.example.university.admin.
2		Blank separator line; it has no runtime effect.
3	`import com.example.university.user.Role;`	Imports com.example.university.user.Role so the controller can use it.
4	`import com.example.university.user.UserAccountService;`	Imports com.example.university.user.UserAccountService so the controller can use it.
5	`import com.example.university.user.UserResponse;`	Imports com.example.university.user.UserResponse so the controller can use it.
6	`import com.example.university.user.UserWriteRequest;`	Imports com.example.university.user.UserWriteRequest so the controller can use it.
7	`import jakarta.validation.Valid;`	Imports jakarta.validation.Valid so the controller can use it.
8	`import org.springframework.http.ResponseEntity;`	Imports org.springframework.http.ResponseEntity so the controller can use it.
9	`import org.springframework.web.bind.annotation.DeleteMapping;`	Imports org.springframework.web.bind.annotation.DeleteMapping so the controller can use it.
10	`import org.springframework.web.bind.annotation.GetMapping;`	Imports org.springframework.web.bind.annotation.GetMapping so the controller can use it.
11	`import org.springframework.web.bind.annotation.PathVariable;`	Imports org.springframework.web.bind.annotation.PathVariable so the controller can use it.
12	`import org.springframework.web.bind.annotation.PostMapping;`	Imports org.springframework.web.bind.annotation.PostMapping so the controller can use it.
13	`import org.springframework.web.bind.annotation.PutMapping;`	Imports org.springframework.web.bind.annotation.PutMapping so the controller can use it.
14	`import org.springframework.web.bind.annotation.RequestBody;`	Imports org.springframework.web.bind.annotation.RequestBody so the controller can use it.
15	`import org.springframework.web.bind.annotation.RequestMapping;`	Imports org.springframework.web.bind.annotation.RequestMapping so the controller can use it.
16	`import org.springframework.web.bind.annotation.RestController;`	Imports org.springframework.web.bind.annotation.RestController so the controller can use it.
17	`import org.springframework.web.servlet.support.ServletUriComponentsBuilder;`	Imports org.springframework.web.servlet.support.ServletUriComponentsBuilder so the controller can use it.
18		Blank separator line; it has no runtime effect.
19	`import java.net.URI;`	Imports java.net.URI so the controller can use it.
20	`import java.util.List;`	Imports java.util.List so the controller can use it.
21		Blank separator line; it has no runtime effect.
22	`@RestController`	Marks this class as a REST controller. Spring writes returned objects as JSON instead of rendering HTML.
23	`@RequestMapping("/api/admins")`	Sets the base REST path to /api/admins for every handler in this class.
24	`public class AdminController {`	Declares the public controller class.
25		Blank separator line; it has no runtime effect.
26	`    private static final Role ROLE = Role.ADMIN;`	Fixes the account role for this controller to Role.ADMIN.
27		Blank separator line; it has no runtime effect.
28	`    private final UserAccountService userAccountService;`	Declares the service dependency that performs user CRUD and password hashing.
29		Blank separator line; it has no runtime effect.

Line	Source line	Explanation
30	`    public AdminController(UserAccountService userAccountService) {`	Constructor injection gives the controller access to the shared user service.
31	`        this.userAccountService = userAccountService;`	Stores the constructor-injected service for use by the HTTP handler methods.
32	`    }`	Closes the preceding Java block.
33		Blank separator line; it has no runtime effect.
34	`    @GetMapping`	Maps HTTP GET on the collection URL to the list method. GET is safe and read-only.
35	`    public ResponseEntity<List<UserResponse>> list() {`	Handler method that returns a list wrapped in ResponseEntity so the HTTP status can be controlled.
36	`        return ResponseEntity.ok(userAccountService.list(ROLE));`	Loads only accounts of this controller's role and returns HTTP 200 with a JSON array.
37	`    }`	Closes the preceding Java block.
38		Blank separator line; it has no runtime effect.
39	`    @GetMapping("/{id}")`	Maps GET /{id} to a single-resource read operation.
40	`    public ResponseEntity<UserResponse> get(@PathVariable Long id) {`	Binds the {id} path segment to the Long id argument and prepares a 200 response.
41	`        return ResponseEntity.ok(userAccountService.get(ROLE, id));`	Finds the item only when both its id and role match, then returns HTTP 200 or 404.
42	`    }`	Closes the preceding Java block.
43		Blank separator line; it has no runtime effect.
44	`    @PostMapping`	Maps HTTP POST on the collection URL to create a new resource.
45	`    public ResponseEntity<UserResponse> create(@Valid @RequestBody UserWriteRequest request) {`	Reads JSON from the request body, validates it, and prepares a response.
46	`UserResponse created = userAccountService.create(ROLE, request);`	Delegates creation to the service, which hashes the password and assigns the controller's role.
47	`URI location = ServletUriComponentsBuilder.fromCurrentRequest()`	Starts building the URL of the new resource from the current request URL.
48	`                .path("/{id}")`	Adds the id path template to the collection URL.
49	`                .buildAndExpand(created.id())`	Substitutes the new id into the path template.
50	`                .toUri();`	Converts the builder result into a java.net.URI used by the Location header.
51	`        return ResponseEntity.created(location).body(created);`	Returns HTTP 201 Created, a Location header, and the created resource as JSON.
52	`    }`	Closes the preceding Java block.
53		Blank separator line; it has no runtime effect.
54	`    @PutMapping("/{id}")`	Maps HTTP PUT /{id} to an update operation.
55	`    public ResponseEntity<UserResponse> update(`	Starts the update handler, which needs an id and a JSON body.
56	`            @PathVariable Long id,`	Reads the id from the path.
57	`            @Valid @RequestBody UserWriteRequest request`	Reads and validates the JSON update body.
58	`    ) {`	Closes the method parameter list and opens the method body.
59	`return ResponseEntity.ok(userAccountService.update(ROLE, id, request));`	Updates only a role-matching account and returns HTTP 200 with the updated JSON.
60	`    }`	Closes the preceding Java block.
61		Blank separator line; it has no runtime effect.
62	`    @DeleteMapping("/{id}")`	Maps HTTP DELETE /{id} to a delete operation.
63	`    public ResponseEntity<Void> delete(@PathVariable Long id) {`	Delete handler takes the path id and returns no response body.
64	`        userAccountService.delete(ROLE, id);`	Deletes the account only if it belongs to this controller's role.
65	`        return ResponseEntity.noContent().build();`	Returns HTTP 204 No Content after a successful delete.
66	`    }`	Closes the preceding Java block.
67	`}`	Closes the preceding Java block.

Teacher REST controller

CRUD for teacher accounts. Both ADMIN and TEACHER roles may use this route.

Line	Source line	Explanation
1	`package com.example.university.teacher;`	Declares the Java package com.example.university.teacher.
2		Blank separator line; it has no runtime effect.
3	`import com.example.university.user.Role;`	Imports com.example.university.user.Role so the controller can use it.
4	`import com.example.university.user.UserAccountService;`	Imports com.example.university.user.UserAccountService so the controller can use it.
5	`import com.example.university.user.UserResponse;`	Imports com.example.university.user.UserResponse so the controller can use it.
6	`import com.example.university.user.UserWriteRequest;`	Imports com.example.university.user.UserWriteRequest so the controller can use it.
7	`import jakarta.validation.Valid;`	Imports jakarta.validation.Valid so the controller can use it.
8	`import org.springframework.http.ResponseEntity;`	Imports org.springframework.http.ResponseEntity so the controller can use it.
9	`import org.springframework.web.bind.annotation.DeleteMapping;`	Imports org.springframework.web.bind.annotation.DeleteMapping so the controller can use it.
10	`import org.springframework.web.bind.annotation.GetMapping;`	Imports org.springframework.web.bind.annotation.GetMapping so the controller can use it.
11	`import org.springframework.web.bind.annotation.PathVariable;`	Imports org.springframework.web.bind.annotation.PathVariable so the controller can use it.
12	`import org.springframework.web.bind.annotation.PostMapping;`	Imports org.springframework.web.bind.annotation.PostMapping so the controller can use it.
13	`import org.springframework.web.bind.annotation.PutMapping;`	Imports org.springframework.web.bind.annotation.PutMapping so the controller can use it.
14	`import org.springframework.web.bind.annotation.RequestBody;`	Imports org.springframework.web.bind.annotation.RequestBody so the controller can use it.
15	`import org.springframework.web.bind.annotation.RequestMapping;`	Imports org.springframework.web.bind.annotation.RequestMapping so the controller can use it.
16	`import org.springframework.web.bind.annotation.RestController;`	Imports org.springframework.web.bind.annotation.RestController so the controller can use it.
17	`import org.springframework.web.servlet.support.ServletUriComponentsBuilder;`	Imports org.springframework.web.servlet.support.ServletUriComponentsBuilder so the controller can use it.
18		Blank separator line; it has no runtime effect.
19	`import java.net.URI;`	Imports java.net.URI so the controller can use it.
20	`import java.util.List;`	Imports java.util.List so the controller can use it.
21		Blank separator line; it has no runtime effect.
22	`@RestController`	Marks this class as a REST controller. Spring writes returned objects as JSON instead of rendering HTML.
23	`@RequestMapping("/api/teachers")`	Sets the base REST path to /api/teachers for every handler in this class.
24	`public class TeacherController {`	Declares the public controller class.
25		Blank separator line; it has no runtime effect.
26	`    private static final Role ROLE = Role.TEACHER;`	Fixes the account role for this controller to Role.TEACHER.
27		Blank separator line; it has no runtime effect.
28	`    private final UserAccountService userAccountService;`	Declares the service dependency that performs user CRUD and password hashing.
29		Blank separator line; it has no runtime effect.
30	`    public TeacherController(UserAccountService userAccountService) {`	Constructor injection gives the controller access to the shared user service.
31	`        this.userAccountService = userAccountService;`	Stores the constructor-injected service for use by the HTTP handler methods.
32	`    }`	Closes the preceding Java block.
33		Blank separator line; it has no runtime effect.
34	`    @GetMapping`	Maps HTTP GET on the collection URL to the list method. GET is safe and read-only.

Line	Source line	Explanation
35	`    public ResponseEntity<List<UserResponse>> list() {`	Handler method that returns a list wrapped in ResponseEntity so the HTTP status can be controlled.
36	`        return ResponseEntity.ok(userAccountService.list(ROLE));`	Loads only accounts of this controller's role and returns HTTP 200 with a JSON array.
37	`    }`	Closes the preceding Java block.
38		Blank separator line; it has no runtime effect.
39	`    @GetMapping("/{id}")`	Maps GET /{id} to a single-resource read operation.
40	`    public ResponseEntity<UserResponse> get(@PathVariable Long id) {`	Binds the {id} path segment to the Long id argument and prepares a 200 response.
41	`        return ResponseEntity.ok(userAccountService.get(ROLE, id));`	Finds the item only when both its id and role match, then returns HTTP 200 or 404.
42	`    }`	Closes the preceding Java block.
43		Blank separator line; it has no runtime effect.
44	`    @PostMapping`	Maps HTTP POST on the collection URL to create a new resource.
45	`    public ResponseEntity<UserResponse> create(@Valid @RequestBody UserWriteRequest request) {`	Reads JSON from the request body, validates it, and prepares a response.
46	`UserResponse created = userAccountService.create(ROLE, request);`	Delegates creation to the service, which hashes the password and assigns the controller's role.
47	`URI location = ServletUriComponentsBuilder.fromCurrentRequest()`	Starts building the URL of the new resource from the current request URL.
48	`                .path("/{id}")`	Adds the id path template to the collection URL.
49	`                .buildAndExpand(created.id())`	Substitutes the new id into the path template.
50	`                .toUri();`	Converts the builder result into a java.net.URI used by the Location header.
51	`        return ResponseEntity.created(location).body(created);`	Returns HTTP 201 Created, a Location header, and the created resource as JSON.
52	`    }`	Closes the preceding Java block.
53		Blank separator line; it has no runtime effect.
54	`    @PutMapping("/{id}")`	Maps HTTP PUT /{id} to an update operation.
55	`    public ResponseEntity<UserResponse> update(`	Starts the update handler, which needs an id and a JSON body.
56	`            @PathVariable Long id,`	Reads the id from the path.
57	`            @Valid @RequestBody UserWriteRequest request`	Reads and validates the JSON update body.
58	`    ) {`	Closes the method parameter list and opens the method body.
59	`return ResponseEntity.ok(userAccountService.update(ROLE, id, request));`	Updates only a role-matching account and returns HTTP 200 with the updated JSON.
60	`    }`	Closes the preceding Java block.
61		Blank separator line; it has no runtime effect.
62	`    @DeleteMapping("/{id}")`	Maps HTTP DELETE /{id} to a delete operation.
63	`    public ResponseEntity<Void> delete(@PathVariable Long id) {`	Delete handler takes the path id and returns no response body.
64	`        userAccountService.delete(ROLE, id);`	Deletes the account only if it belongs to this controller's role.
65	`        return ResponseEntity.noContent().build();`	Returns HTTP 204 No Content after a successful delete.
66	`    }`	Closes the preceding Java block.
67	`}`	Closes the preceding Java block.

Student REST controller

CRUD for student accounts. Both ADMIN and STUDENT roles may use this route.

Line	Source line	Explanation
1	`package com.example.university.student;`	Declares the Java package com.example.university.student.
2		Blank separator line; it has no runtime effect.
3	`import com.example.university.user.Role;`	Imports com.example.university.user.Role so the controller can use it.
4	`import com.example.university.user.UserAccountService;`	Imports com.example.university.user.UserAccountService so the controller can use it.
5	`import com.example.university.user.UserResponse;`	Imports com.example.university.user.UserResponse so the controller can use it.
6	`import com.example.university.user.UserWriteRequest;`	Imports com.example.university.user.UserWriteRequest so the controller can use it.
7	`import jakarta.validation.Valid;`	Imports jakarta.validation.Valid so the controller can use it.
8	`import org.springframework.http.ResponseEntity;`	Imports org.springframework.http.ResponseEntity so the controller can use it.
9	`import org.springframework.web.bind.annotation.DeleteMapping;`	Imports org.springframework.web.bind.annotation.DeleteMapping so the controller can use it.
10	`import org.springframework.web.bind.annotation.GetMapping;`	Imports org.springframework.web.bind.annotation.GetMapping so the controller can use it.
11	`import org.springframework.web.bind.annotation.PathVariable;`	Imports org.springframework.web.bind.annotation.PathVariable so the controller can use it.
12	`import org.springframework.web.bind.annotation.PostMapping;`	Imports org.springframework.web.bind.annotation.PostMapping so the controller can use it.
13	`import org.springframework.web.bind.annotation.PutMapping;`	Imports org.springframework.web.bind.annotation.PutMapping so the controller can use it.
14	`import org.springframework.web.bind.annotation.RequestBody;`	Imports org.springframework.web.bind.annotation.RequestBody so the controller can use it.
15	`import org.springframework.web.bind.annotation.RequestMapping;`	Imports org.springframework.web.bind.annotation.RequestMapping so the controller can use it.
16	`import org.springframework.web.bind.annotation.RestController;`	Imports org.springframework.web.bind.annotation.RestController so the controller can use it.
17	`import org.springframework.web.servlet.support.ServletUriComponentsBuilder;`	Imports org.springframework.web.servlet.support.ServletUriComponentsBuilder so the controller can use it.
18		Blank separator line; it has no runtime effect.
19	`import java.net.URI;`	Imports java.net.URI so the controller can use it.
20	`import java.util.List;`	Imports java.util.List so the controller can use it.
21		Blank separator line; it has no runtime effect.
22	`@RestController`	Marks this class as a REST controller. Spring writes returned objects as JSON instead of rendering HTML.
23	`@RequestMapping("/api/students")`	Sets the base REST path to /api/students for every handler in this class.
24	`public class StudentController {`	Declares the public controller class.
25		Blank separator line; it has no runtime effect.
26	`    private static final Role ROLE = Role.STUDENT;`	Fixes the account role for this controller to Role.STUDENT.
27		Blank separator line; it has no runtime effect.
28	`    private final UserAccountService userAccountService;`	Declares the service dependency that performs user CRUD and password hashing.
29		Blank separator line; it has no runtime effect.
30	`    public StudentController(UserAccountService userAccountService) {`	Constructor injection gives the controller access to the shared user service.
31	`        this.userAccountService = userAccountService;`	Stores the constructor-injected service for use by the HTTP handler methods.
32	`    }`	Closes the preceding Java block.
33		Blank separator line; it has no runtime effect.
34	`    @GetMapping`	Maps HTTP GET on the collection URL to the list method. GET is safe and read-only.

Line	Source line	Explanation
35	`    public ResponseEntity<List<UserResponse>> list() {`	Handler method that returns a list wrapped in ResponseEntity so the HTTP status can be controlled.
36	`        return ResponseEntity.ok(userAccountService.list(ROLE));`	Loads only accounts of this controller's role and returns HTTP 200 with a JSON array.
37	`    }`	Closes the preceding Java block.
38		Blank separator line; it has no runtime effect.
39	`    @GetMapping("/{id}")`	Maps GET /{id} to a single-resource read operation.
40	`    public ResponseEntity<UserResponse> get(@PathVariable Long id) {`	Binds the {id} path segment to the Long id argument and prepares a 200 response.
41	`        return ResponseEntity.ok(userAccountService.get(ROLE, id));`	Finds the item only when both its id and role match, then returns HTTP 200 or 404.
42	`    }`	Closes the preceding Java block.
43		Blank separator line; it has no runtime effect.
44	`    @PostMapping`	Maps HTTP POST on the collection URL to create a new resource.
45	`    public ResponseEntity<UserResponse> create(@Valid @RequestBody UserWriteRequest request) {`	Reads JSON from the request body, validates it, and prepares a response.
46	`UserResponse created = userAccountService.create(ROLE, request);`	Delegates creation to the service, which hashes the password and assigns the controller's role.
47	`URI location = ServletUriComponentsBuilder.fromCurrentRequest()`	Starts building the URL of the new resource from the current request URL.
48	`                .path("/{id}")`	Adds the id path template to the collection URL.
49	`                .buildAndExpand(created.id())`	Substitutes the new id into the path template.
50	`                .toUri();`	Converts the builder result into a java.net.URI used by the Location header.
51	`        return ResponseEntity.created(location).body(created);`	Returns HTTP 201 Created, a Location header, and the created resource as JSON.
52	`    }`	Closes the preceding Java block.
53		Blank separator line; it has no runtime effect.
54	`    @PutMapping("/{id}")`	Maps HTTP PUT /{id} to an update operation.
55	`    public ResponseEntity<UserResponse> update(`	Starts the update handler, which needs an id and a JSON body.
56	`            @PathVariable Long id,`	Reads the id from the path.
57	`            @Valid @RequestBody UserWriteRequest request`	Reads and validates the JSON update body.
58	`    ) {`	Closes the method parameter list and opens the method body.
59	`return ResponseEntity.ok(userAccountService.update(ROLE, id, request));`	Updates only a role-matching account and returns HTTP 200 with the updated JSON.
60	`    }`	Closes the preceding Java block.
61		Blank separator line; it has no runtime effect.
62	`    @DeleteMapping("/{id}")`	Maps HTTP DELETE /{id} to a delete operation.
63	`    public ResponseEntity<Void> delete(@PathVariable Long id) {`	Delete handler takes the path id and returns no response body.
64	`        userAccountService.delete(ROLE, id);`	Deletes the account only if it belongs to this controller's role.
65	`        return ResponseEntity.noContent().build();`	Returns HTTP 204 No Content after a successful delete.
66	`    }`	Closes the preceding Java block.
67	`}`	Closes the preceding Java block.

11. Request and response DTOs: line by line

DTOs define the JSON contract at the API boundary. Requests are validated before controller logic runs; responses are deliberately shaped so sensitive fields such as the password hash cannot leak.

LoginRequest.java

Line	Source line	Explanation
1	`package com.example.university.auth;`	Declares the authentication DTO package.
2		Blank separator line.
3	`import jakarta.validation.constraints.NotBlank;`	Imports jakarta.validation.constraints.NotBlank for this API data model.
4		Blank separator line.
5	`public record LoginRequest(`	Defines LoginRequest as an immutable Java record used for the JSON login body.
6	`        @NotBlank(message = "Username is required") String username,`	Username field; @NotBlank rejects null, empty, and whitespace-only values.
7	`        @NotBlank(message = "Password is required") String password`	Password field; @NotBlank rejects missing or blank credentials.
8	`) {`	Closes the record parameter list and opens its body.
9	`}`	Closes the record; no extra methods are needed.

LoginResponse.java

Line	Source line	Explanation
1	`package com.example.university.auth;`	Declares the authentication DTO package.
2		Blank separator line.
3	`import com.example.university.user.UserResponse;`	Imports com.example.university.user.UserResponse for this API data model.
4		Blank separator line.
5	`public record LoginResponse(`	Defines LoginResponse as an immutable JSON response record.
6	`        String accessToken,`	The signed JWT the client must send in the Authorization header.
7	`        String tokenType,`	The token type, which is Bearer in this application.
8	`        long expiresIn,`	Token lifetime in seconds.
9	`        UserResponse user`	Safe user details, including role, without the password hash.
10	`) {`	Closes the record parameter list and opens its body.
11	`}`	Closes the record.

UserWriteRequest.java

Line	Source line	Explanation
1	`package com.example.university.user;`	Declares the user DTO package.
2		Blank separator line.
3	`import jakarta.validation.constraints.Email;`	Imports jakarta.validation.constraints.Email for this API data model.
4	`import jakarta.validation.constraints.NotBlank;`	Imports jakarta.validation.constraints.NotBlank for this API data model.
5	`import jakarta.validation.constraints.Pattern;`	Imports jakarta.validation.constraints.Pattern for this API data model.
6	`import jakarta.validation.constraints.Size;`	Imports jakarta.validation.constraints.Size for this API data model.
7		Blank separator line.
8	`public record UserWriteRequest(`	Defines the JSON body used by both POST and PUT user endpoints.
9	`        @NotBlank(message = "Username is required")`	Username is required.
10	`        @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")`	Username length must be between 3 and 50 characters.
11	`        @Pattern(`	Starts the username format constraint.
12	`                regexp = "^[A-Za-z0-9._-]+$",`	Allows only letters, numbers, dot, underscore, and hyphen.
13	`                message = "Username may only contain letters, numbers, dots, underscores, and hyphens"`	Validation message returned when the username format is invalid.
14	`        )`	Closes the @Pattern annotation.
15	`        String username,`	The username property.
16		Blank separator line.
17	`        @Size(min = 8, max = 72, message = "Password must be between 8 and 72 characters")`	If a password is supplied, it must be 8 to 72 characters; null means keep the old password on PUT.
18	`        String password,`	The password property. It is hashed before storage and never returned.
19		Blank separator line.
20	`        @NotBlank(message = "Full name is required")`	Full name is required.
21	`        @Size(max = 100, message = "Full name must be at most 100 characters")`	Full name maximum length.
22	`        String fullName,`	The full name property.
23		Blank separator line.
24	`        @NotBlank(message = "Email is required")`	Email is required.
25	`        @Email(message = "Email must be valid")`	Email must have a valid email format.
26	`        @Size(max = 150, message = "Email must be at most 150 characters")`	Email maximum length.
27	`        String email,`	The email property.
28		Blank separator line.
29	`        Boolean enabled`	Optional enabled flag; null defaults to true on create and preserves the value on update.
30	`) {`	Closes the record parameter list and opens its body.
31	`}`	Closes the record.

UserResponse.java

Line	Source line	Explanation
1	`package com.example.university.user;`	Declares the user response package.
2		Blank separator line.
3	`import java.time.Instant;`	Imports java.time.Instant for this API data model.
4		Blank separator line.
5	`public record UserResponse(`	Defines the public JSON shape for a user returned by the REST API.
6	`        Long id,`	Database identifier.
7	`        String username,`	Normalized username.
8	`        String fullName,`	Display name.
9	`        String email,`	Normalized email address.
10	`        Role role,`	ADMIN, TEACHER, or STUDENT.
11	`        boolean enabled,`	Whether the account may authenticate.
12	`        Instant createdAt,`	Creation timestamp.
13	`        Instant updatedAt`	Last update timestamp.
14	`) {`	Closes the record header and opens the body.
15		Blank separator line.
16	`    public static UserResponse from(User user) {`	Static factory that converts the JPA User entity into a safe response DTO.
17	`        return new UserResponse(`	Creates a new UserResponse.
18	`                user.getId(),`	Copies the id.
19	`                user.getUsername(),`	Copies the username.
20	`                user.getFullName(),`	Copies the full name.
21	`                user.getEmail(),`	Copies the email.
22	`                user.getRole(),`	Copies the role.
23	`                user.isEnabled(),`	Copies the enabled flag.
24	`                user.getCreatedAt(),`	Copies createdAt.
25	`                user.getUpdatedAt()`	Copies updatedAt.
26	`        );`	Closes the constructor call.
27	`    }`	Closes the factory method.
28	`}`	Closes the UserResponse record.

ApiError.java

Line	Source line	Explanation
1	`package com.example.university.error;`	Declares the error DTO package.
2		Blank separator line.
3	`import jakarta.servlet.http.HttpServletRequest;`	Imports jakarta.servlet.http.HttpServletRequest for this API data model.
4		Blank separator line.
5	`import java.time.Instant;`	Imports java.time.Instant for this API data model.
6	`import java.util.Map;`	Imports java.util.Map for this API data model.
7		Blank separator line.
8	`public record ApiError(`	Defines the consistent JSON body used for REST API errors.
9	`        Instant timestamp,`	UTC timestamp of the error.
10	`        int status,`	Numeric HTTP status code.
11	`        String error,`	Short HTTP reason phrase.
12	`        String message,`	Human-readable error message.
13	`        String path,`	Request URI where the error occurred.
14	`        Map<String, String> fieldErrors`	Optional map of field name to validation message.
15	`) {`	Closes the record header and opens the body.
16		Blank separator line.
17	`public static ApiError of(HttpServletRequest request, int status, String error, String message) {`	Factory for general errors that do not have field-level validation details.
18	`return new ApiError(Instant.now(), status, error, message, request.getRequestURI(), null);`	Creates the error with the current time, supplied status, message, path, and null field errors.
19	`    }`	Closes the general error factory.
20		Blank separator line.
21	`    public static ApiError validation(HttpServletRequest request, Map<String, String> fieldErrors) {`	Factory for request-validation errors.
22	`        return new ApiError(`	Creates a 400 Bad Request error.
23	`                Instant.now(),`	Uses the current timestamp.
24	`                400,`	Sets status to 400.
25	`                "Bad Request",`	Sets the HTTP reason phrase.
26	`                "Request validation failed",`	Sets a general validation message.
27	`                request.getRequestURI(),`	Copies the request path.
28	`                fieldErrors`	Attaches field-level validation messages.
29	`        );`	Closes the validation error constructor call.
30	`    }`	Closes the validation factory.
31	`}`	Closes the ApiError record.

12. REST security route rules: line by line

These SecurityConfig lines are the RBAC boundary for the REST API. They run before controller code, so an unauthorized request never reaches the endpoint method.

Line	Source line	Explanation
47	`.authorizeHttpRequests(authorize -> authorize`	Starts the URL authorization rules.
48	`.requestMatchers("/api/auth/login").permitAll()`	Allows anyone to call the login endpoint.
49	`.requestMatchers("/actuator/health", "/actuator/health/**", "/actuator/info").permitAll()`	Allows unauthenticated health/info monitoring endpoints.
50	`.requestMatchers("/error").permitAll()`	Allows Spring Boot's error dispatch endpoint.
51	`.requestMatchers("/api/admins/**").hasRole("ADMIN")`	Only ADMIN may access /api/admins/**.
52	`.requestMatchers("/api/teachers/**").hasAnyRole("ADMIN", "TEACHER")`	ADMIN and TEACHER may access /api/teachers/**.
53	`.requestMatchers("/api/students/**").hasAnyRole("ADMIN", "STUDENT")`	ADMIN and STUDENT may access /api/students/**.
54	`.anyRequest().authenticated()`	Any other endpoint requires a valid authenticated token.
55	`)`	Closes the authorization-rule block.
56	`.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);`	Adds the JWT filter before form-based username/password authentication processing.

Meaning of hasRole vs hasAnyRole: hasRole('ADMIN') requires authority ROLE_ADMIN. hasAnyRole('ADMIN','TEACHER') accepts either ROLE_ADMIN or ROLE_TEACHER. The JWT filter creates the authority from the role currently stored in the database.

13. JWT request filter: line by line

JwtAuthenticationFilter is where the API reads the Bearer token. It validates the signature and expiry, reloads the user from the database, rejects revoked tokens, and puts the authenticated principal into Spring Security's context.

Line	Source line	Explanation
1	`package com.example.university.security;`	Declares the security package.
2		Blank separator line.
3	`import com.example.university.auth.UserPrincipal;`	Imports com.example.university.auth.UserPrincipal for JWT request authentication.
4	`import com.example.university.user.User;`	Imports com.example.university.user.User for JWT request authentication.
5	`import com.example.university.user.UserRepository;`	Imports com.example.university.user.UserRepository for JWT request authentication.
6	`import io.jsonwebtoken.Claims;`	Imports io.jsonwebtoken.Claims for JWT request authentication.
7	`import io.jsonwebtoken.JwtException;`	Imports io.jsonwebtoken.JwtException for JWT request authentication.
8	`import jakarta.servlet.FilterChain;`	Imports jakarta.servlet.FilterChain for JWT request authentication.
9	`import jakarta.servlet.ServletException;`	Imports jakarta.servlet.ServletException for JWT request authentication.
10	`import jakarta.servlet.http.HttpServletRequest;`	Imports jakarta.servlet.http.HttpServletRequest for JWT request authentication.
11	`import jakarta.servlet.http.HttpServletResponse;`	Imports jakarta.servlet.http.HttpServletResponse for JWT request authentication.
12	`import org.springframework.http.HttpHeaders;`	Imports org.springframework.http.HttpHeaders for JWT request authentication.
13	`import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;`	Imports org.springframework.security.authentication.UsernamePasswordAuthenticationToken for JWT request authentication.
14	`import org.springframework.security.core.authority.SimpleGrantedAuthority;`	Imports org.springframework.security.core.authority.SimpleGrantedAuthority for JWT request authentication.
15	`import org.springframework.security.core.context.SecurityContextHolder;`	Imports org.springframework.security.core.context.SecurityContextHolder for JWT request authentication.
16	`import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;`	Imports org.springframework.security.web.authentication.WebAuthenticationDetailsSource for JWT request authentication.
17	`import org.springframework.stereotype.Component;`	Imports org.springframework.stereotype.Component for JWT request authentication.
18	`import org.springframework.web.filter.OncePerRequestFilter;`	Imports org.springframework.web.filter.OncePerRequestFilter for JWT request authentication.
19		Blank separator line.
20	`import java.io.IOException;`	Imports java.io.IOException for JWT request authentication.
21	`import java.util.List;`	Imports java.util.List for JWT request authentication.
22	`import java.util.Optional;`	Imports java.util.Optional for JWT request authentication.
23		Blank separator line.
24	`@Component`	Registers this filter as a Spring component.
25	`public class JwtAuthenticationFilter extends OncePerRequestFilter {`	Declares the JWT authentication filter.
26		Blank separator line.
27	`    private static final String BEARER_PREFIX = "Bearer ";`	Defines the required Authorization header prefix.
28		Blank separator line.
29	`    private final JwtService jwtService;`	Injects the JWT parser/generator service.
30	`    private final UserRepository userRepository;`	Injects the repository used to load the token's user fresh from the database.
31		Blank separator line.
32	`public JwtAuthenticationFilter(JwtService jwtService, UserRepository userRepository) {`	Starts constructor injection.
33	`        this.jwtService = jwtService;`	Stores the JWT service.
34	`        this.userRepository = userRepository;`	Stores the user repository.
35	`    }`	Closes the constructor.
36		Blank separator line.
37	`    @Override`	Overrides the once-per-request filter method.
38	`    protected void doFilterInternal(`	Starts the filter method.

Line	Source line	Explanation
39	`            HttpServletRequest request,`	Receives the HTTP request.
40	`            HttpServletResponse response,`	Receives the HTTP response.
41	`            FilterChain filterChain`	Receives the next filter in the chain.
42	`    ) throws ServletException, IOException {`	Declares the checked exceptions thrown by the filter.
43	`String authorization = request.getHeader(HttpHeaders.AUTHORIZATION);`	Reads the Authorization header from the request.
44	`if (authorization == null || !authorization.startsWith(BEARER_PREFIX)) {`	Checks that a Bearer token is present.
45	`            filterChain.doFilter(request, response);`	Continues the filter chain when no token exists.
46	`            return;`	Returns immediately from this filter.
47	`        }`	Closes the no-token branch.
48		Blank separator line.
49	`String token = authorization.substring(BEARER_PREFIX.length()).trim();`	Extracts the token text after 'Bearer ' and trims whitespace.
50	`if (token.isEmpty() || SecurityContextHolder.getContext().getAuthentication() != null) {`	Skips processing for an empty token or when an authentication already exists.
51	`            filterChain.doFilter(request, response);`	Continues without changing the security context.
52	`            return;`	Returns from the filter.
53	`        }`	Closes the skip branch.
54		Blank separator line.
55	`        try {`	Starts token parsing and validation.
56	`            Claims claims = jwtService.parseClaims(token);`	Verifies the JWT signature/expiry and returns its claims.
57	`            String username = claims.getSubject();`	Reads the username stored in the JWT subject.
58	`            Object versionClaim = claims.get("ver");`	Reads the token-version claim used for logout revocation.
59		Blank separator line.
60	`            if (username != null && versionClaim instanceof Number tokenVersion) {`	Requires a username and a numeric version claim.
61	`                Optional<User> candidate = userRepository.findByUsername(username);`	Loads the user by username from the database.
62	`                if (candidate.isPresent()) {`	Checks whether the user still exists.
63	`                    User user = candidate.get();`	Gets the current database user.
64	`                    if (user.isEnabled() && user.getTokenVersion() == tokenVersion.longValue()) {`	Rejects disabled accounts and tokens issued before the last logout/password change.
65	`UserPrincipal principal = new UserPrincipal(user.getId(), user.getUsername(), user.getRole());`	Builds the safe principal stored in Spring Security.
66	`UsernamePasswordAuthenticationToken authentication =`	Starts creating an authenticated Spring Security token.
67	`new UsernamePasswordAuthenticationToken(`	Passes the principal and no password credentials.
68	`                                    principal,`	Sets the principal.
69	`                                    null,`	No password is stored in the authentication object.
70	`                                    List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole().name()))`	Grants the ROLE_* authority based on the current database role.
71	`                                );`	Closes the authentication token constructor.
72	`authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));`	Attaches web request details.
73	`SecurityContextHolder.getContext().setAuthentication(authentication);`	Publishes the authentication to the current request's security context.
74	`                    }`	Closes the role/permission block.
75	`                }`	Closes the user-present block.
76	`            }`	Closes the username/version block.
77	`        } catch (JwtException | IllegalArgumentException ignored) {`	Catches malformed, expired, or invalid JWTs.
78	`            SecurityContextHolder.clearContext();`	Clears any partial authentication so the request remains unauthenticated.
79	`        }`	Closes the try/catch block.
80		Blank separator line.
81	`        filterChain.doFilter(request, response);`	Continues the filter chain; protected routes will now be allowed or rejected by authorization rules.
82	`    }`	Closes the filter method.
83	`}`	Closes the JwtAuthenticationFilter class.

Why token version is checked: Logout increments the account's token version. Password changes and enabled-state changes also increment it. Old JWTs remain cryptographically valid, but the version mismatch makes them unusable.

14. Supporting REST classes and HTTP status summary

UserAccountService: methods used by REST controllers

Method	Lines	Role in REST API
list(Role)	Lines 23-27	Returns only accounts matching the requested role for GET collection.
get(Role, Long)	Lines 29-31	Returns one role-matching account for GET /{id}.
getById(Long)	Lines 33-36	Used by /api/auth/me.
create(Role, request)	Lines 38-58	Checks password presence and length, rejects duplicates, hashes the password, assigns the endpoint role, and saves.
update(Role, id, request)	Lines 60-86	Updates role-matching account, optionally rehashes password, and increments token version on security-sensitive changes.
delete(Role, id)	Lines 88-92	Deletes only an account matching the endpoint role.
revokeTokens(Long)	Lines 94-99	Increments token version for logout.

GlobalExceptionHandler: exception to HTTP mapping

Exception	Response	Reason
ResourceNotFoundException	404 Not Found	Missing account or role mismatch.
UsernameAlreadyExistsException	409 Conflict	Duplicate username.
InvalidRequestException	400 Bad Request	Business validation failure.
InvalidCredentialsException	401 Unauthorized	Bad login.
MethodArgumentNotValidException	400 Bad Request	Bean Validation field errors; returns fieldErrors map.
HttpMessageNotReadableException / type mismatch	400 Bad Request	Malformed JSON or invalid path value.
DataIntegrityViolationException	409 Conflict	Database conflict.
NoResourceFoundException	404 Not Found	Unknown URL.
HttpRequestMethodNotSupportedException	405 Method Not Allowed	Unsupported HTTP verb.
HttpMediaTypeNotSupportedException	415 Unsupported Media Type	Wrong Content-Type.
Exception (fallback)	500 Internal Server Error	Unexpected error; details are logged, not returned.

Final REST API summary

The REST API surface consists of four controllers. All three resource controllers use the same service and DTOs, which gives consistent JSON and status behavior. Spring Security evaluates role rules before controller dispatch, and JwtAuthenticationFilter turns a Bearer token into the authenticated principal and ROLE_* authority.

- Admin: can call every CRUD route.
- Teacher: can call only /api/teachers/** plus authentication endpoints.
- Student: can call only /api/students/** plus authentication endpoints.
- Passwords are BCrypt-hashed and never serialized in responses.
- Logout revokes all previously issued tokens for the account.

End of document: This PDF was generated from the actual project files and line numbers. Rebuild the project with .\mvnw.cmd test to verify the behavior after any future edits.